

AIMLCZG546 · SESSION 2 · COMPANION READER

ML in Production

From Research Prototype to Production System

Session 2 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas and diagrams. This reader gives you the *why*. Every concept is expanded in plain English. Every slide example is worked out step by step. Five colour-coded boxes guide you. **Blue** = the core idea in one breath. **Amber** = a real-life analogy before the math. **Green** = a worked example with real numbers. **Red** = the common mistake to avoid. **Purple** = the one line to remember.

1 What this session covers

This session answers one big question: what happens to a machine learning model after the research phase ends?

Here are the five topics we cover, in order:

1. **The production gap** — why a good model is only the start.
2. **ML model vs ML system** — two very different ways to think about your work.
3. **ML and non-ML components** — how ML fits into the larger software picture.
4. **AI paradigms** — predictive, generative, and agentic AI compared.
5. **Case studies** — Apollo (Baidu) and Microsoft show these ideas in real systems.

2 The Production Gap: Sidney's Startup

Here is a story that plays out over and over in industry. A researcher builds a great model in a notebook. The test-set accuracy looks strong. Everyone is excited. Then they try to deploy it to real users — and that is where the real work begins.

★ Everyday picture

Think of a chef who perfects a recipe at home. It works beautifully for six guests. Then a restaurant chain wants to serve it to 10,000 people per day. The recipe is only a tiny part of the problem. The real challenges are sourcing ingredients, training staff, quality control, and managing costs. Moving from *recipe* to *restaurant* takes far more engineering than cooking the dish.

 The intuition

A **production gap** is the gap between a working research model and a working production system. The model is just the starting point. The system around it takes as much work as the model itself. That system includes data pipelines, serving infrastructure, monitoring, teams, and cost control.

In the slide's case study, **Sidney** builds a speech recognition model. In the lab it hits 94% accuracy. In production he runs into seven problems:

- **Noisy data** — real call audio has background noise and accents not in the training set.
- **Latency** — the model takes 3 seconds per clip; the product needs under 0.5 seconds.
- **Cost** — running the model 24/7 at scale costs 40× the research estimate.
- **Fragile pipelines** — the preprocessing script breaks when the call centre upgrades its format.
- **Team gaps** — ML researchers and software engineers use different tools and vocabulary.
- **Monitoring** — accuracy drifts for three weeks before anyone notices.
- **Fairness** — the model is 15% worse on non-native English speakers.

 Worked example: diagnosing Sidney's 3-second latency

Sidney's model takes 3 seconds per audio clip. The product requires 0.5 seconds. Let us trace the bottleneck.

Step 1. Measure each stage. Total pipeline time: 3.0 s. Preprocessing (resampling, noise filter): 1.8 s. Model inference: 0.9 s. Postprocessing (text formatting): 0.3 s.

Step 2. Find the biggest cost. Preprocessing takes 1.8 s — 60% of the total.

Step 3. Optimise the top item. Replace the pure-Python noise filter with a compiled C library. New preprocessing time: 0.2 s.

Step 4. Re-measure. New total: $0.2 + 0.9 + 0.3 = 1.4$ s. Still over budget.

Step 5. Optimise inference. Quantise the model from 32-bit to 8-bit weights. New inference time: 0.25 s.

Step 6. Final total. $0.2 + 0.25 + 0.3 = 0.75$ s. Within budget.

The bottleneck was *not* the model — it was the preprocessing step. A system-centric view found the fix faster.

 Watch out

Do not assume the model is the bottleneck. In production systems, data pipelines and infrastructure are often the slow parts. Profile first, then optimise.

✓ Key takeaway

Moving from research prototype to production system needs as much engineering as building the model — often more. Profile the whole system before you tune the model.

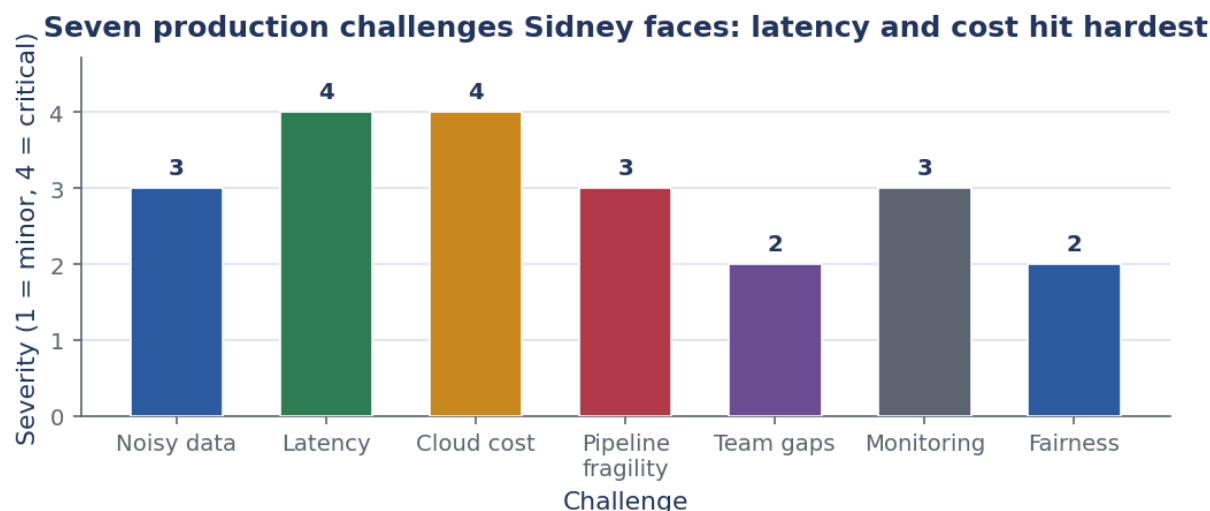


Figure 1: Seven production challenges that Sidney’s startup faces. Latency and cloud cost are the most severe. Each bar is one challenge that the model alone cannot fix.

Where this shows up in ML. Every deployed ML product — a recommendation engine, a fraud detector, a language model in a product — faces this gap. **MLOps** (ML Operations) is the discipline that bridges it. It covers automated pipelines, model registries, monitoring dashboards, and continuous retraining.

3 ML Model vs ML System

There are two very different ways to think about an ML project.

🔍 The intuition

An **ML model** is the trained function that maps inputs to outputs. An **ML system** is the entire product. It includes data pipelines, the model, serving infrastructure, monitoring, feedback loops, and the team that runs all of it. The model is one component of the system.

★ Everyday picture

An ML model is like the engine in a car. A better engine helps. But the car’s usefulness also depends on the chassis, the steering, the fuel system, and the dashboard. Tuning only the engine while ignoring everything else is *model-centric* thinking.

3.1 Model-Centric View

A **model-centric** team asks: “How do we make the model more accurate?” They tune hyperparameters, try new architectures, and add labelled data. Progress stops when the model tops out.

3.2 System-Centric View

A **system-centric** team asks: “How do we make the whole system better for users?” The model is one part of a larger system. Gains can come from cleaner data, faster serving, better monitoring, or a better user interface — not just the model.

3.3 The Iceberg of Hidden Technical Debt

Sculley et al. (2015) studied ML systems at Google. They found that the ML model code is only a small fraction of the total codebase. The rest is infrastructure.

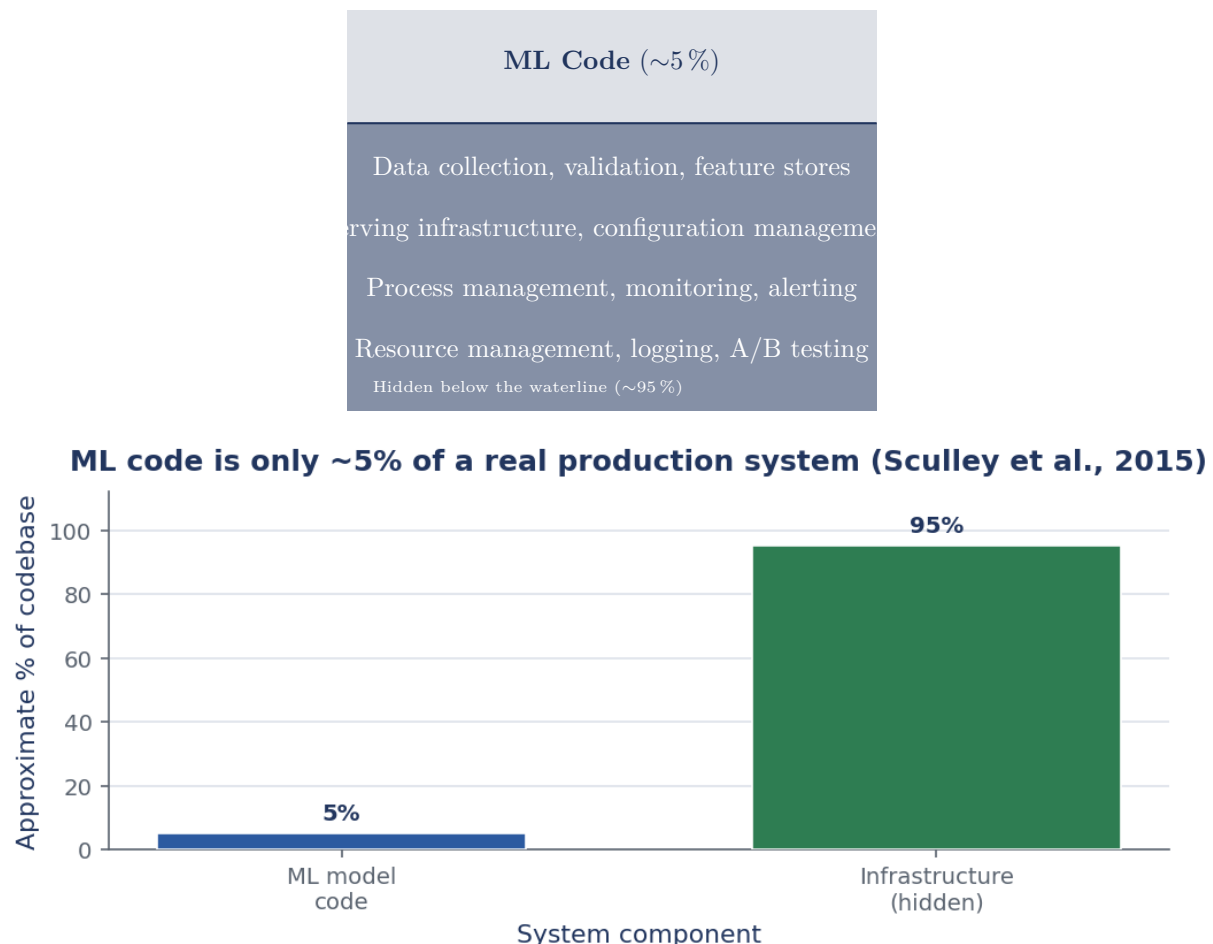


Figure 2: ML model code is about 5 % of a production system. The other 95 % is data work, infrastructure, monitoring, and configuration. Focus on the whole iceberg, not just the tip.

⚠ Watch out

Optimising the ML model while ignoring the infrastructure is like polishing an engine while the fuel tank leaks. The fuel tank will fail first.

🔗 Worked example: model-centric vs system-centric fraud detection

A bank's fraud model has 88 % precision: 12 % of blocked transactions are legitimate. Two teams try to fix it.

Step 1. Model-centric team. Collect more labelled fraud examples. Retrain with XGBoost. New precision: 90 %. Time: 3 weeks.

Step 2. System-centric team. Study why false positives occur. Discovery: the feature pipeline uses yesterday's exchange rates for international payments. Fix the pipeline (a two-line code change). New precision: 93%. Time: 2 days.

Step 3. Compare. The system-centric team got a larger gain in less time. They found the bug by looking *beyond* the model.

The system-centric view wins here because the root cause was a data pipeline bug, not a model weakness.

Where this shows up in ML. Industry data shows that in most ML projects, 80% of the time goes to data work (collection, cleaning, labelling). Only 20% goes to model building. This is the **80/20 rule of ML engineering**.

✓ Key takeaway

The ML model is a small part of the ML system. System-centric thinking — looking at data, infrastructure, and monitoring — usually finds bigger wins faster.

4 ML and Non-ML Components in a System

Every production ML product is a mix of ML code and traditional software. The ML component may be the core of the product or just one supporting feature.

🔍 The intuition

ML is a *component*, not a product. A transcription service uses ML as its core value. A tax software product uses ML as one feature (audit risk flagging) alongside many non-ML components. The ratio of ML to non-ML code shapes the engineering challenges.

★ Everyday picture

Think of a car assembly line. The engine (ML model) is important. But the line also needs conveyor belts, robotic arms, quality sensors, and human inspectors. The non-ML parts must be designed to work reliably with the ML part.

Here is how ML fits in three real products:

Product	ML role	Non-ML components
Speech transcription	Core (the product is the ML output)	Audio codec, API gateway, billing
Tax audit risk tool	Auxiliary (flags for human review)	Form parser, rule engine, tax database
E-commerce recommendations	Core (drives revenue)	Product catalogue, cart, payment system

Systems thinking means you must design every non-ML component around the ML component's failure modes. When the model makes a mistake, who sees it? What happens next? How do you recover? These are software engineering questions, not ML questions.

🔧 Worked example: what happens when the speech model fails

The transcription service misidentifies a medical term ("hypertension" as "high tension").

Step 1. Detection. The monitoring system flags a spike in user corrections.

Step 2. Diagnosis. Log analysis shows the error rate is higher for audio with background

noise above 40 dB.

Step 3. Fix options. Option A: retrain the model on noisy-audio examples (weeks). Option B: add a noise-detection pre-filter that routes noisy clips to a different model (days).

Step 4. Rollback. While the fix is prepared, the system routes flagged clips to human review. This is a non-ML fallback — a software engineering design decision made before the model was deployed.

The non-ML fallback (human review routing) kept the product usable while the ML fix was built. This is why you design the full system, not just the model.

✗ Watch out

A product that relies 100 % on the ML model with no non-ML fallback will fail visibly when the model makes a mistake. Always design for graceful degradation.

Where this shows up in ML. The field of **responsible AI** requires non-ML checks: bias audits, fairness dashboards, human-in-the-loop review, and audit logs. These are all non-ML components that wrap the ML model.

✓ Key takeaway

ML is one component in a larger system. Design the whole system — including how the non-ML parts handle model errors — before you deploy.

ML component share by product type: some products are mostly ML, others are not

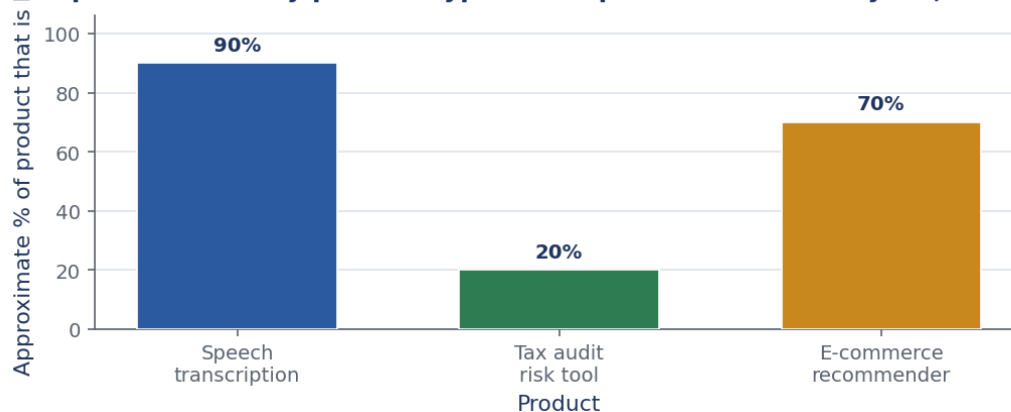


Figure 3: How much of each product is ML. Speech transcription is almost entirely ML. Tax audit risk flagging is mostly non-ML. E-commerce recommendations are mostly ML. The mix changes the engineering approach.

5 The ML Pipeline: From Data to Deployment

A **machine learning pipeline** is the sequence of steps that turns raw data into a running model. In research, the pipeline is mostly a one-time process. In production, the pipeline is a cycle that never stops.

The 7-stage ML pipeline: data work dominates stages 1-4

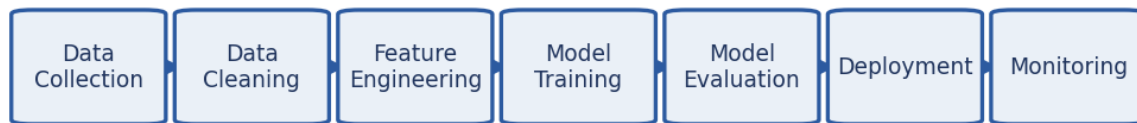


Figure 4: The 7-stage ML pipeline. Stages 1–4 are all data work. In production, the pipeline is a loop: new data feeds back into training, and the deployed model feeds back into monitoring.

The intuition

Think of the pipeline as a factory assembly line. Raw material (data) comes in at one end. A finished product (deployed model) comes out the other end. Each station on the line transforms the material. If any station breaks, the factory stops.

The key stages are:

1. **Data collection** — gather labelled examples from real-world sources.
2. **Data cleaning** — remove noise, fix errors, handle missing values.
3. **Feature engineering** — turn raw data into numbers the model can learn from.
4. **Model training** — fit the model to the training set.
5. **Model evaluation** — check performance on a held-out set.
6. **Deployment** — serve the model to real users via an API.
7. **Monitoring** — watch for data drift, accuracy drops, and cost spikes.

In production, Stage 7 loops back to Stage 1. New production data is collected, cleaned, and used to retrain the model. This cycle is called **continuous training**.

Watch out

A static pipeline (train once, deploy forever) will degrade over time. Real-world data changes: new slang, new products, new user behaviours. A model trained last year on last year's data will make mistakes on this year's inputs.

Where this shows up in ML. **MLOps** tools (Kubeflow, MLflow, Airflow, Vertex AI Pipelines) automate this cycle. They version every dataset, every model, and every experiment. They trigger retraining when monitoring detects drift.

Key takeaway

The ML pipeline is a loop, not a line. Monitoring and retraining are as important as training itself.

6 AI Paradigms: Predictive, Generative, and Agentic

Modern AI systems fall into three paradigms. Each has different capabilities and different engineering requirements.

The intuition

Predictive AI answers a question: “Will this customer churn?”

Generative AI creates something new: “Write me an email.”

Agentic AI achieves a goal: “Book me a flight, write the agenda, and send the invite.”

Each paradigm is more powerful and more complex than the last.

6.1 Predictive AI

Predictive AI uses past data to forecast future outcomes. You train a model on historical examples. The model answers a fixed question: a label, a number, or a probability. Output: one answer per input.

Examples: spam filter, fraud detector, churn predictor, demand forecast.

6.2 Generative AI

Generative AI creates new content from patterns it learned. A **foundation model** (a large model pre-trained on huge data) is given a natural-language instruction. It produces a digital artefact: text, code, an image, or a video.

The slide defines it precisely: “A system based on a foundation model that generates digital artefacts based on natural user instructions.” The key word is *generates* — the output is new, not just a category.

6.3 Agentic AI

Agentic AI acts on its own to reach a goal. It can reason: plan multi-step actions. It can use tools: call APIs, search the web, run code. It can reflect: check its own output and try again.

The slide gives three definitions that all point to the same idea:

Source	Definition
Nvidia	Uses reasoning and planning to solve complex, multi-step problems.
OpenAI	Pursues complex goals with limited direct supervision.
Schneider (2025)	A foundation-model system that plans, reflects, and uses tools.

The key formula from the slide:

$$\text{Agentic AI} = \text{Generative AI} + \text{Reasoning} + \text{Tools} + \text{Feedback Loops}$$

★ Everyday picture

Predictive AI is a weather-forecast app: you ask one question, it gives one answer. Generative AI is a ghostwriter: you say “draft the email” and it writes the email. Agentic AI is a personal assistant. You say “handle the trip.” It books the flight, writes the agenda, sends the invite, and revises the plan when the venue changes.

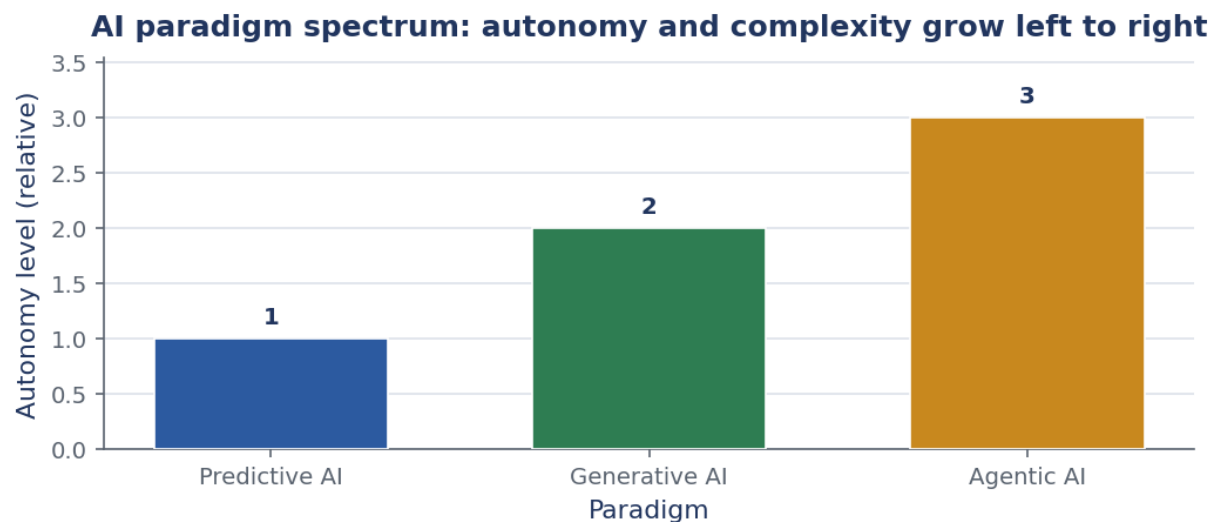


Figure 5: The three AI paradigm types ranked by autonomy. Predictive AI answers one question. Generative AI creates content. Agentic AI pursues a goal through a sequence of actions. Each step up adds more autonomy and more engineering complexity.

Worked example: classifying three AI products

Three products are described. Classify each as predictive, generative, or agentic.

Step 1. Product A: “Given a customer’s purchase history, predict whether they will buy again this month.” Output: a probability (one number). The model answers a fixed question. **Classification: Predictive AI.**

Step 2. Product B: “Given a product name and description, write three marketing headlines.” Output: new text. The system creates content. **Classification: Generative AI.**

Step 3. Product C: “Find the cheapest flight to Singapore next week. Book it. Apply my loyalty points and confirm the booking.” The system searches databases, compares prices, and takes action. Output: a completed multi-step action. The system plans and acts. **Classification: Agentic AI.**

The key test: does it answer one question (predictive), create content (generative), or plan and act on its own (agentic)?

Watch out

Agentic AI is not just a smarter chatbot. It can take real-world actions: send emails, make purchases, delete files. This changes the SE challenges. You must design safety guardrails, rollback mechanisms, and human checkpoints.

Where this shows up in ML. Most production ML products today are predictive. Generative AI products are growing fast. Agentic AI is the next frontier: it requires new SE practices (agent orchestration, tool safety, audit logs, and human oversight).

Key takeaway

Predictive AI answers one question. Generative AI creates content. Agentic AI achieves a goal. Each step up the ladder adds more power and more engineering complexity.

7 Case Study: Apollo Autonomous Driving (Baidu)

Baidu's Apollo platform is one of the most complex ML systems ever built. It shows what “ML system” means at industrial scale.

The intuition

Apollo is a self-driving car platform. It uses **28 ML models** working together in real time. No single model does all the work. Each model handles one task and passes its output to the next model. The whole pipeline runs 30 times per second.

★ Everyday picture

Think of a surgery team. The anaesthetist, surgeon, and scrub nurse each do one specialised job. Their outputs feed into each other. If one person makes an error, it affects everyone downstream. Apollo's 28 models work the same way.

Apollo's models cover three areas:

- **Perception** — cameras and LiDAR see lanes, vehicles, pedestrians, and signs.
- **Detection and classification** — models label each object and predict its type.
- **Trajectory prediction** — models forecast where each object will be in the next 3–5 seconds.

Apollo's 7-stage perception pipeline: 28 ML models work in sequence

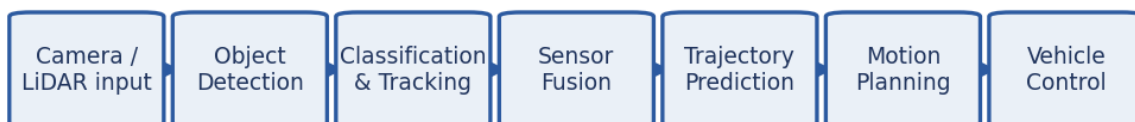


Figure 6: Apollo's 7-stage perception pipeline. Each box is a model or a processing step. The output of one box is the input to the next. A miss at any stage (for example, failing to detect a pedestrian) causes errors in all later stages.

Worked example: tracing a cascading failure in Apollo

A camera frame arrives. The Object Detection model misses one pedestrian (a false negative).

Step 1. Detection stage. Object Detection outputs 13 bounding boxes (missed the pedestrian).

Step 2. Classification stage. The 13 boxes are classified. The pedestrian is not in any box.

Step 3. Sensor fusion. LiDAR points from the pedestrian are present, but the fusion model has no detection to attach them to. The pedestrian is not in the 3D map.

Step 4. Trajectory prediction. No pedestrian in the 3D map. No trajectory is predicted.

Step 5. Motion planning. The planner sees no obstacle. It does not slow down or steer around the person.

Step 6. Outcome. The vehicle continues at speed. This is a safety-critical failure caused by one false negative in Step 1.

This is a **cascading failure**: one error in an early model propagates through all later models. It shows why SE practices (redundancy, sensor fusion, safety stops) are as important as model accuracy.

SE challenges in Apollo:

- **High complexity.** 28 interacting models must all be tested together.
- **Low test coverage.** Probabilistic models are hard to unit-test.
- **Model maintenance.** Each of the 28 models must be versioned and retrained separately.
- **Safety requirements.** A wrong prediction can harm a person.

Where this shows up in ML. Apollo illustrates all four SE challenges from the lecture. Multi-source data fusion, model interaction design, and safety-critical testing are active research areas in ML systems engineering.

✓ Key takeaway

A real ML system is not one model. It is many models working together. A single model's mistake can cascade through the whole system. SE practices — redundancy, fallbacks, and testing — protect against this.

8 Case Study: Microsoft's 9-Stage ML Workflow

Microsoft studied how software teams build ML products. They found a consistent 9-stage workflow and key lessons about how ML development differs from traditional software.

🧠 The intuition

Traditional software development has a plan: gather requirements, design, code, test, deploy. ML development has a loop: gather requirements, get data, clean data, label data, engineer features, train, evaluate, deploy, monitor. Then the loop starts again. The big difference is that *data* is the central engineering activity, not code.

Microsoft's 9-stage ML workflow: stages 1-5 are all data work



Figure 7: Microsoft's 9-stage ML workflow. Stages 1–5 (left half) are all about data. Only Stage 6 (model training) and Stage 7 (evaluation) are traditional ML. Stages 8 and 9 (deployment and monitoring) connect the model to production.

Stage	Name	Key question
1	Model Requirements	What problem are we solving?
2	Data Collection	Where does the data come from?
3	Data Cleaning	Is the data accurate and complete?
4	Data Labelling	Do we have ground truth? Who labels it?
5	Feature Engineering	What signals will help the model?
6	Model Training	What algorithm and hyperparameters?
7	Model Evaluation	Does it meet the performance target?
8	Deployment	How is the model served at scale?
9	Monitoring	Is the model still working well?

★ Everyday picture

Think of building a house. Stage 1 is the blueprint. Stages 2–5 are all the groundwork: digging foundations, laying pipes, pouring concrete. Stage 6 is framing the walls. Stage 7 is the inspection. Stages 8–9 are moving in and maintaining the house. Most people see only the framing (Stage 6). The groundwork (Stages 2–5) is what takes the time.

🔧 Worked example: why data labelling slows down real teams (Stage 4)

A team wants to train a model to detect defects in factory photos.

Step 1. Collect images. The factory cameras produce 10,000 photos per day. Storing one week of data: 70,000 images.

Step 2. Sample for labelling. The team needs 5,000 labelled images to train. They randomly sample 5,000 from the 70,000.

Step 3. Label the images. Each image needs a human expert to mark defect regions. Time per image: 2 minutes. Total: $5,000 \times 2 = 10,000$ minutes = 167 hours. With two labellers working 8 hours per day: $167 \div 16 \approx 10$ working days.

Step 4. Quality check. A second labeller re-checks 10 % of labels. Disagreement rate: 8 %. Those 400 images need a third labeller to adjudicate.

Step 5. Total cost. Two full-time labellers for two weeks, plus domain-expert time for

adjudication. This is *before* any model training begins.
 This is why Microsoft found that data work (Stages 2–4) dominates ML project timelines.

Key findings from Microsoft’s research:

- **Data is the core.** Teams spend more time on Stages 2–5 than on Stages 6–7.
- **Iteration is the rule.** Teams run dozens of experiments before picking a model.
- **Cross-functional teams are needed.** ML engineers, data engineers, domain experts, and product managers must work together from Stage 1.
- **Versioning is not optional.** Both models and datasets must be versioned.

⚠ Watch out

“We’ll fix the data later” is one of the most expensive sentences in ML. Data quality problems found after model training require going back to Stage 3 or 4. That means re-cleaning, re-labelling, and retraining — weeks of work lost. Fix data quality before you train.

Where this shows up in ML. Data-centric AI (Andrew Ng’s term) is the practice of treating data quality as the primary engineering activity. Tools like **Label Studio**, **Scale AI**, and **Snorkel** automate parts of the labelling and cleaning stages.

✓ Key takeaway

In the Microsoft workflow, model training is Stage 6 out of 9. Four data stages come before it. “Data first, model second” is the industrial reality.

Glossary & Symbols

Key terms.

production gap	the gap between a working research model and a working production system.
ML system	the full product: data pipelines, model, serving, monitoring, and team.
ML model	the trained function that maps inputs to outputs; one part of the ML system.
model-centric	a mindset that focuses only on improving the model’s accuracy.
system-centric	a mindset that looks at the whole system to find the best improvement.
ML pipeline	the sequence of steps from raw data to a deployed model.
continuous training	automatically retraining the model as new production data arrives.
MLOps	practices and tools for running ML systems reliably in production.
predictive AI	AI that uses past data to answer a fixed question (label, number, probability).
generative AI	AI that creates new content (text, image, code) from a natural instruction.

agentic AI AI that plans and takes sequences of real-world actions to achieve a goal.

foundation model

a large model pre-trained on huge data; fine-tuned for specific tasks.

cascading failure

one model's error propagates through all downstream models.

data-centric AI the practice of treating data quality as the primary engineering activity.

Symbols at a glance.

Symbol / term	Meaning
~5 % ML code	the fraction of a production system that is model code
~95 % hidden	the fraction that is infrastructure (Sculley et al., 2015)
28 models	number of ML models in Apollo (Baidu's autonomous driving system)
9 stages	Microsoft's ML workflow: 5 data stages + training + eval + deploy + monitor
Stage 4	data labelling: usually the most time-consuming stage